

МЕТОДЫ И ИНСТРУМЕНТЫ ТЕСТИРОВАНИЯ ПРИЛОЖЕНИЙ

Тестирование программ и систем

Тестирование программ и систем - это способ семантической проверки программы, который заключается в обработке программой последовательности разнообразных контрольных наборов тестов с известными результатами. Тесты подбираются так, чтобы они охватили самые разнообразные типы возможных ситуаций.

Тест - это совокупность входных данных и/или действий пользователя с указанием ожидаемых результатов или соответствующих реакций программы, которая предназначена для проверки работоспособности программы и выявления в ней ошибочных ситуаций.

Тестовую проверку можно провести также путем добавления к программе, проверяется, дополнительных операторов, которые будут сигнализировать о ходе ее выполнения и получения результатов.

Тестовые данные, предназначены для проверки работы системы, создаются по-разному: генератором тестовых данных, проектной группой на основе документов или файлов, пользователем из спецификации требований и т.д. Очень часто разрабатываются специальные формы входных документов, в которых отражается процесс выполнения программы с помощью тестовых данных.

Виды тестирования программ с целью проверки:

- полноты функций;
- согласованности интерфейсов;
- структуры программы;
- исчисления и корректности выполнения функций;
- правильности функционирования в заданных условиях;
- надежности выполнения программ;
- эффективности защиты от сбоев аппаратуры и необнаруженных ошибок;
- удобства применения и сопровождения.

Много типов тестов готовит сам заказчик для проверки работы ИС. Структура и содержание тестов зависят от вида элемента - модуль, компонент, группа компонент, подсистема или система. Некоторые тесты, связанные с необходимостью проверить, работает ИС согласно проекту, удовлетворены требования заказчика.

Верификация – проверка продукта на соответствие входным данным, правилам, стандартам.

Валидация – проверка продукта на соответствие потребностям пользователя

Процесс тестирования

1. Планирование и управление (Test Management)
2. Проектирование тестов (Test Design)
3. Проведение тестирования (Test Execution)
4. Анализ полученных результатов (Test Analysis)
5. Создание отчетов

Цели тестирования

1. Обнаружение дефектов
2. Повышение уверенности в качестве продукта
3. Предоставление информации для принятия решений
4. Предупреждение дефектов

Основные виды работ по тестированию:

- верификация результатов разработки программного продукта на каждом этапе жизненного цикла;
- составление плана тестирования и подготовки тестов для проверки отдельных элементов разработанной программы и программы в целом;
- управление выполнением тестов и анализ результатов тестирования;
- повторное тестирование.

Конечной целью тестирования промышленных ИТ-проектов является получение сертификата на разработанный программный продукт.

Методы тестирования

По автоматизации выполнения

- Ручное тестирование
- Автоматизированное тестирование

По степени подготовки к тестированию

- По заранее подготовленной документации
- Исследовательское
- Ad-hoc

По позитивности

- Позитивное
- Негативное

По исполнению кода

- Статическое
 - Анализ кода
 - Тестирование требований
 - Тестирование макетов и прототипов
 - Тестирование пользовательской документации
- Динамическое

По знанию системы

- Черного ящика
- Белого ящика
- Серого ящика

Методы тестирования и верификации целиком зависят от методов проектирования и стадий, из которых начинается проверка правильности функционирования результатов проектирования.

Статические методы используются во время проведения инспекций и анализа спецификаций компонентов без их выполнения, а динамические применяются в процессе выполнения программ.

Тесты, основанные на внешних спецификациях программного обеспечения, применяемые на этапе комплексного тестирования для определения полноты решения функциональных задач и их соответствия исходным требованиям.

Функциональному тестированию предшествует анализ функций, в задачи которого входят:

- идентификация множества функциональных требований;
- идентификация внешних функций в реализации программного обеспечения и построение последовательностей функций соответственно использованию их в ПО;
- идентификация множества входных данных каждой функции и определение направлений их изменения;
- построение тестовых наборов и сценариев тестирования функций;
- выявление и представление всех функциональных требований с помощью тестовых наборов и проведения тестирования ошибок в программе и взаимодействия со средой.

Основная цель тестирования - обеспечение полноты и согласованности реализованных в программных компонентах функций и интерфейсов между ними.

Методы доказательства правильности программ появились еще в 80-е годы. Техника символьного выполнения включает моделирование выполнения кода, используя символы вместо переменных данных.

Автоматизированное тестирование — процесс верификации программного обеспечения, при котором основные функции и шаги теста, такие как запуск, инициализация, выполнение, анализ и выдача результата, выполняются автоматически при помощи инструментов для автоматизированного тестирования.

Уровни автоматизированного тестирования. Автоматизированное тестирование может проводиться на любом уровне тестирования: компонентном, интеграционном, системном.

GUI-тесты

Это тесты системного уровня, которые работают с приложением как с «черным ящиком». Они взаимодействуют с приложением через графический пользовательский интерфейс — нажимают на кнопки, заполняют значения в полях ввода и т.д. Такие тесты нестабильны (легко ломаются при любом изменении интерфейса), часто дают ложные срабатывания, дороги в написании и поддержке. Но эти тесты позволяют проверить, что система выполняет необходимые пользовательские сценарии.

Интеграционные тесты

Это тесты, которые проверяют интеграцию различных частей системы. Они позволяют убедиться, что все компоненты системы запущены, каналы работоспособны, форматы взаимодействия соответствуют спецификациям. Эти тесты довольно легко создавать, но поддержка может отнимать больше времени, когда продукт находится в фазе активной разработки, и требования к подсистемам меняются. Тем не менее они крайне полезны для контроля целостности системы — после обновления какого-то из модулей можно быстро убедиться, что он работает так, как ожидается. Интеграционные тесты очень редко дают ложные срабатывания и выполняются намного быстрее приемочных тестов, но все же медленнее модульных тестов.

Модульные тесты

Это тесты, которые проверяют поведение отдельного класса (или нескольких тесно взаимосвязанных классов). Они не взаимодействуют с какими-либо внешними интерфейсами. Модульные тесты довольно дешевы в написании и поддержке, быстро выполняются и дают моментальную обратную связь.

Пирамида тестирования

Для повышения эффективности автоматизированного тестирования полезно придерживаться так называемой «**пирамиды тестирования**». Смысл ее заключается в том, что наибольшее количество тестов должно приходиться на компонентное (модульное тестирование), чуть меньшее — на интеграционное, и на самом верху, на системном уровне, имеет смысл автоматизировать весьма ограниченное количество сценариев.



Плюсы данного подхода:

1. Раннее обнаружение ошибок
2. Автоматизация происходит в основном через программные интерфейсы
3. Сравнительно легко поддерживать
4. Сравнительно невысокая стоимость

Программные средства отслеживания ошибок, автоматизации процесса тестирования и системы непрерывной интеграции приведены в таблице.

Таблица - Программные средства для тестирования ПО

Системы отслеживания ошибок свободно распространяемые	Системы отслеживания ошибок требующие оплаты лицензии
Redmine, BUGS — the Bug Genie	Atlassian JIRA
Bugzilla, eTraxis	Bontq
GNATS, Mantis bug tracking system	PVCS Tracker

Trac, Em Forge	Project Kaiser
Picket; Flyspray, D EVP ROM YouTrack	TrackStudio Enterprise

Приложения для автоматизации тестирования:

HP LoadRunner, HP QuickTest Professional, HP Quality Center Segue Silk Performer

IBM Rational FunctionalTester, IBM Rational PerformanceTester

IBM Rational TestStudio

Smart Bear Software TestComplete

Системы непрерывной интеграции BuildBot, Hudson или Jenkins, FinalBuilder, TeamCity

Уровни тестирования

Компонентное тестирование (также известное как модульное) занимается поиском дефектов и верификацией функционирования программных модулей, программ, объектов, классов и т.п., которые можно протестировать изолированно. Это может быть сделано изолированно от остальной части системы, в зависимости от контекста жизненного цикла разработки и системы. В процессе могут быть использованы заглушки, драйвера и эмуляторы. Компонентное тестирование может включать как тестирование функциональности и специфичных нефункциональных характеристик, таких как поведение ресурсов (например, поиск утечки памяти) или тестирование надежности, так и структурное тестирование (например, покрытие кода).

Обычно, компонентное тестирование производится с доступом к тестируемому коду и с поддержкой рабочего окружения, такого как фреймворк модульного тестирования или утилиты отладки. На практике компонентное тестирование обычно производится разработчиками, которые пишут код. Дефекты обычно исправляются сразу после того, как становятся известны, без занесения их в базу дефектов. Один из подходов к компонентному тестированию – составить автоматизированные тестовые сценарии до кодирования. Это называется разработкой, управляемой тестированием. Этот подход состоит из множества итераций и основывается на циклах разработки тестовых сценариев, написании и интеграции небольшого участка кода, и выполнении компонентного тестирования, корректируя любые проблемы и выполняя тесты, пока не будет получен положительный результат.

Интеграционное тестирование проверяет интерфейсы между компонентами, взаимодействие различных частей системы, таких как операционная системы, файловая система, аппаратное обеспечение, и интерфейсы между системами.

Интеграционное тестирование может состоять из одного или более уровней и может быть выполнено на тестовых объектах разного размера следующим образом:

1. Компонентное интеграционное тестирование проверяет взаимодействие между программными компонентами и производится после компонентного тестирования
2. Системное интеграционное тестирование проверяет взаимодействие между системами или между аппаратным обеспечением и может быть выполнено после системного тестирования. В этом случае, разработчики могут управлять только одной стороной интерфейса. Однако, это может рассматриваться как риск. Бизнес-процессы могут включать последовательность систем; могут быть важны кроссплатформенные различия.

Чем больше объем интеграции, тем труднее становится изолировать дефекты отдельного компонента или системы, которые могут привести к увеличению рисков и требующих дополнительного времени для решения проблем. Стратегии системного интеграционного тестирования могут основываться на архитектуре системы (такой как нисходящая или восходящая), функциональных задачах, последовательности обработки транзакций или других аспектах системы и ее компонентов. Для того, чтобы упростить процесс изоляции отказа и как можно раньше обнаруживать дефекты, интеграция должна проводиться по возрастающей, а не происходить по сценарию «большого взрыва». Тестирование специфичных нефункциональных характеристик (например, производительности), может быть включено в интеграционное тестирование, наравне с функциональным. На каждой стадии интеграции тестировщики

концентрируют все внимание именно на интеграции как таковой. Например, если интегрируется модуль А с модулем В, они проверяют взаимодействие модулей, а не функциональность каждого из них, так как она должна быть проверена во время компонентного тестирования. Для тестирования могут использоваться как функциональный, так и структурный подходы. В идеале, тестировщики должны понимать архитектуру и ее влияние на интеграционное планирование.

Если интеграционное тестирование планируется до разработки компонентов или системы, эти компоненты могут разрабатываться в порядке, обеспечивающем наиболее эффективное тестирование.

Системное тестирование сконцентрировано на поведении тестового объекта как целостной системы или продукта.

Во время системного тестирования тестовое окружение должно быть как можно ближе к предполагаемому эксплуатационному окружению системы для минимизации риска пропуска отказов, связанных с эксплуатационным окружением системы. Системное тестирование может включать тесты, основанные на рисках или спецификациях требований, бизнес-процессах, сценариях использования системы, или других высокоуровневых текстовых описаниях или моделях поведения системы, взаимодействия с ОС и системными ресурсами. Системное тестирование должно заниматься исследованием функциональных и нефункциональных требований к системе и качеством обрабатываемых данных. Тестировщики также должны уметь выполнять свои обязанности в случае неполных или недокументированных требований.

Системное тестирование функциональных требований начинается с тестирования на основе спецификаций (тестирования методом черного ящика), для различных аспектов системы. Например, таблица решений может быть создана на основе бизнес-правил работы системы.

- **Альфа-тестирование** - моделируемое или действительное эксплуатационное тестирование потенциальными пользователями/заказчиками или независимой командой тестирования на стороне разработчиков, но вне разрабатывающей организации. Альфа-тестирование часто применяется к коробочному программному обеспечению в качестве внутреннего приёмочного тестирования.

- **Приемочное тестирование** - формальное тестирование по отношению к потребностям, требованиям и бизнес процессам пользователя, проводимое с целью определения соответствия системы критериям приёмки и дать возможность пользователям, заказчикам или иным авторизованным лицам определить, принимать систему или нет.

- **Бета-тестирование** - эксплуатационное тестирование потенциальными и/или существующими клиентами/заказчиками на внешней стороне с целью определения действительно ли компонент или система удовлетворяет требованиям клиента/заказчика и вписывается в бизнес-процессы. Бета-тестирование часто проводится как форма внешнего приёмочного тестирования готового программного обеспечения для того чтобы получить отзывы рынка.

Виды тестирования

Тестирование функциональности

1. **Smoke-тестирование.** Выполняется, как правило, сразу после получения билда на тестирование, чтобы убедиться, что наиболее важные части и функции приложения работоспособны. Цель данного вида тестирования – сразу же «вернуть» неработоспособное приложение на исправление. Пример типичных smoke-тестов:

- Установка приложения;
- Открытие/сохранение файлов;
- Загрузка сайта;
- Регистрация/вход на сайт;
- Базовая функциональность приложения;

2. **Sanity-тестирование.** Выполняется после получения нового билда с незначительными изменениями, чтобы удостовериться, что эти изменения присутствуют в билде, а также убедиться в общей «адекватности» билда. Данный вид тестирования позволяет не тратить зря время на тестирование новой функциональности, если новый билд собран неверно или исправление содержит критические ошибки.

Smoke	Sanity
Позволяет убедиться, что наиболее важная функциональность приложения работает корректно	Позволяет убедиться, что новая функциональность добавлена или баги исправлены
Цель тестирования – проверить стабильность системы и ее пригодность для более тщательного тестирования	Цель тестирования – проверить адекватность системы и ее пригодность для более тщательного тестирования
Выполняется тестировщиками или разработчиками	Обычно выполняется тестировщиками
Как правило, задокументировано и выполняется по сценариям	Обычно не задокументировано, строгие сценарии не определены
Подмножество регрессионного тестирования	Подмножество тестирования новой функциональности/подтверждающего
Выполняется на всей системе и проверяет общее состояние системы	Выполняется только над определенной частью системы и проверяет состояние части системы

3. **Тестирование критического пути** – тестирование функциональности, используемой наиболее часто в повседневной деятельности типичных пользователей.
4. **Тестирование новой функциональности**
5. **Подтверждающее тестирование**
6. **Регрессионное тестирование**

Тестирование производительности

- **Нагрузочное.** Тестирование компонента или системы под постоянно увеличивающейся нагрузкой (число одновременно работающих пользователей, число транзакций и т.п.) для определения максимально допустимого уровня нагрузки для исследуемого компонента или системы.
- **Стресс.** Тестирование для оценки системы или компонента на граничных значениях рабочих нагрузок или за их пределами, или в состоянии ограниченных ресурсов, таких как память, процессор, доступ к серверу, ширина пропускного канала и пр.
- **Стабильности.** Тестирование системы или компонента в течение продолжительного времени на средних рабочих нагрузках. Одна из основных целей тестирования – определение корректности использования ресурсов.
- **Масштабируемости.** Исследование способности приложения увеличивать показатели производительности в соответствии с увеличением количества доступных приложению ресурсов.

Тестирование совместимости

Тестирование продукта на разных окружениях, как аппаратных, так и программных.

Совместимость с аппаратным обеспечением

К конфигурационному тестированию на аппаратном уровне можно отнести проверку работы системы со следующим оборудованием:

1. Процессор (Intel/AMD, x86/ARM, 32/64bit etc)
2. Видеокарта
3. Жесткий диск (скорость, тип и пр.)
4. Устройства вывода изображения
 - a. Монитор (способ подключения, разрешение, цветопередача и др.)
 - b. Экран ноутбука/планшета (разрешение, количество точек на дюйм)

- c. Проектор (разрешение, способ подключения)
- d. Несколько мониторов (ноутбук+монитор, монитор+проектор и пр.)
- e. и др.
- 5. Устройства ввода/мапуляторы
 - a. Клавиатура (мультимедиа и др. спецклавиши, отсутствие NumPad, отсутствие правого Alt/Ctrl, ..)
 - b. Мышь, трэкпад, трэкболл и пр. (Разрешение, точность, спецклавиши и пр.)
 - c. Джойстики
 - d. и др.
- 6. Прочие периферийные устройства
 - a. CD/DVD приводы (способ подключения, скорость, возможность записи, поддерживаемые типы дисков и пр.)
 - b. Веб-камеры (внешние/встроенные, по способу подключения, по разрешающей способности и пр.)
 - c. Графические планшеты
 - d. Принтеры
 - e. Сканеры
 - f. Микрофоны
 - g. Наушники
 - h. и др.
- 7. Сетевое оборудование
 - a. Свитч
 - b. Роутер
 - c. Репитер
 - d. 3G/4G-модем
 - e. Различные каналы связи
 - f. и др.

Совместимость с программным обеспечением

- 1. Операционные системы
 - a. Windows XP/7/8/10
 - b. Mac OS
 - c. Linux
 - d. iOS
 - e. Android
 - f. Windows Phone
 - g. Прочие *nix
 - h. Прочие мобильные (MeeGo, Firefox OS, Symbian,...)
 - i. Прочие desktop (ChromeOS, ...)
- 2. Браузеры
 - a. IE 8/9/10/11/Edge
 - b. Chrome
 - c. Firefox
 - d. Прочие Chromium-based: Opera, Yandex и пр.
 - e. Safari
 - f. Safari-iOS
 - g. Android Browser
 - h. IE Mobile
- 3. ПО для безопасности
 - a. Антивирус
 - b. Файрвол
 - c. Трафик-инспектор
- 4. Runtime Environment
 - a. .NET
 - b. Java

5. Драйверы
6. Кодеки
7. Веб-сервер
 - a. Apache
 - b. NGINX
 - c. IIS
 - d. Tomcat
8. Базы данных
 - a. MySQL,
 - b. MSSQL,
 - c. Postgres,
 - d. NoSQL
 - e. и пр.
9. Прочее ПО

Для чего проводить конфигурационное тестирование?

1. Определить минимально допустимое и оптимальное окружения для ПО
2. Убедиться, что ПО корректно работает на основных допустимых конфигурациях:

как правило, проверить все возможные конфигурации невозможно, поэтому для проведения тестирования надо выделить набор наиболее вероятных конфигураций ПО.

Юзабилити

Интерфейса

Безопасности

- Конфиденциальность
- Целостность
- Доступность

Интернационализации и локализации

- Корректность текстов
- Расположение текстов
- Даты и время
- Разделители целой/дробной части
- Обозначение валют
- Нелатинские символы в пути к приложению
- Нелатинские символы в имени пользователя
- Иероглифы
- RTL
- И пр.

Доступности

- Совместимость с программами чтения с экрана
- Управление только с клавиатуры